Parecer — Proposta B: Extrair o Catálogo para um microsserviço

Mateus Vitor Ferreira · Sistemas de Informação, UFLA
Manutenção e Evolução de Software · Prof. Heitor Augustus Xavier Costa

Repositório do projeto: https://github.com/mateus-vitor-ferreira-dev/compra-facil

Recomendação: adotar a **Proposta B**, com duas condições: extração **gradual**, com o catálogo novo convivendo com o módulo antigo até ser validado, e **capacitação da equipe** em sistemas distribuídos antes de ampliar a separação para outras partes do sistema.

1. Por que essa proposta faz sentido para a CompraFácil?

O gargalo é localizado. Nas campanhas, as consultas ao catálogo crescem muito mais que pedidos, estoque, pagamentos e notificações. No monolito, a única saída é ampliar os recursos da aplicação inteira, pagando para escalar justamente o que não está sob pressão. Com o catálogo separado, os recursos vão para onde está a demanda: mais réplicas do catálogo durante a campanha, e o restante como está.

Os times já são dois, mas a implantação é uma só. Um grupo cuida do catálogo e das campanhas, o outro do processamento de compras, e os dois coordenam as alterações numa implantação conjunta. As campanhas são exatamente o momento em que o time do catálogo precisa mudar preços, vitrines e regras com rapidez. Com o catálogo como serviço próprio, esse time passa a implantar no seu ritmo, sem esperar o calendário do outro.

O cenário permite fazer isso com segurança. O código já tem módulos identificáveis, então o catálogo tem uma fronteira natural para ser extraído. A equipe também tem testes automatizados e um processo de implantação confiável, que são a base para validar o serviço novo ao lado do antigo. O ponto fraco do cenário, a pouca experiência com sistemas distribuídos, é o motivo das duas condições da recomendação: começar por **um único serviço**, de leitura predominante e com poucas dependências, limita o tamanho do aprendizado e do risco.

2. Benefícios e limitações

Benefícios

1. **Escala independente, com custo proporcional ao pico.** Num experimento com a mesma carga de campanha (pico de 1.000 requisições/s, 90% leituras de catálogo), escalar **só o catálogo** de 1 para 3 réplicas levou o p95 de **686 ms para 5 ms**. O custo foi **+0,27 núcleo de CPU** e +14 MiB de memória, sem tocar nos demais serviços. Num cluster Kubernetes, o ajuste é automático: o

autoescalonador levou o catálogo de 1 para 4 réplicas durante o pico e de volta a 1 depois, com 0% de erros. No monolito, o mesmo ganho exigiria replicar a aplicação inteira.

2. **Autonomia de entrega do time de catálogo.** Com pipeline, contrato e banco próprios, uma mudança só no catálogo publica só o catálogo, sem esperar o calendário do outro time. No projeto, o lead time mediano do catálogo, do primeiro commit à imagem publicada, ficou em torno de 25 minutos.

Limitações

1. **A integração deixa de ser uma chamada de método.** O processamento de compras passa a depender do catálogo pela rede: preço e disponibilidade precisam de **contrato versionado**, prazo, novas tentativas e um plano para quando o catálogo estiver fora (por exemplo, uma cópia local dos dados de produto, mantida por eventos). Os dados de produto passam a ter dono, e quem os consome lida com consistência eventual.
2. **Mais custo operacional e de capacitação.** Um serviço a mais significa mais um banco, pipeline, painel de monitoramento e ponto de falha. Investigar um problema passa a exigir **observabilidade distribuída** (traces que atravessam serviços, métricas por serviço), e a equipe precisa aprender isso. Parte do acoplamento também persiste: bibliotecas e contratos compartilhados ainda forçam implantações conjuntas. No projeto, só 1 de 5 implantações do catálogo saiu sozinha; as outras vieram de mudanças comuns a todos os serviços (pipeline, observabilidade).

3. Como verificar se a escolha foi adequada?

Critério 1 — Escala independente no pico. Acompanhar, durante as campanhas e em testes de carga que as simulam, o **p95 e a taxa de erros das consultas ao catálogo**, junto com o **CPU e a memória totais** gastos para atendê-las. A escolha se confirma se, escalando apenas o catálogo, o p95 fica dentro da meta (por exemplo, abaixo de 300 ms) com erros abaixo de 1%, e se o recurso total gasto for menor que o de escalar a aplicação inteira para o mesmo resultado. Esses números vêm de um teste de carga reproduzível e do painel de monitoramento do catálogo.

Critério 2 — Autonomia de entrega. Acompanhar, pelo histórico do pipeline, **quantas implantações por semana cada time faz**, o **lead time** (da alteração à produção) e, principalmente, a **fração de implantações do catálogo que saíram sem coordenação** com o time de compras. A escolha se confirma se o time de catálogo passar a implantar com mais frequência e com lead time menor do que no calendário conjunto, e se a maioria das suas implantações sair sozinha. Se muitas implantações continuarem conjuntas, é sinal de que a fronteira entre os serviços ou os contratos precisam ser revistos.

Evidências do projeto

Os números acima vêm da implementação de referência deste repositório, que foi além da Proposta B e separou também o processamento de compras, para exercitar os mecanismos que a extração exige (contratos, eventos, SAGA, observabilidade):

- **Escala independente:** resultados de carga (`task experiment`).
- **Autonomia de entrega:** resultados de entrega (`scripts/delivery_metrics.py`).
- **Escala automática:** Kubernetes local (`task k8s:load`): o HPA levou o catálogo de 1 para 4 réplicas e de volta a 1.
- **Catálogo fora do ar sem derrubar as compras:** `task chaos:catalog`, com 0 falhas em 1.202 pedidos.
- **Integração segura:** contrato consumer-driven do orders verificado pelo catálogo antes de cada publicação (ADR 0004) e teste ponta a ponta da compra no CI.
- **Um limite observado:** com o catálogo escalado, o gargalo do pico passou para o pagamento, que processa um evento por vez. Escalar uma parte desloca o problema para a seguinte, por isso o critério 1 olha o sistema, não só o serviço extraído.
- **Arquitetura e decisões:** diagramas C4 e ADRs.